

LoRaWAN-Based Emergency Alert System for the Elderly with Fall Detection and an Emergency Button

Fadhlan Bani Nugraha

Location	Panti Sosial Fakku Raqabah, an elderly care home in Bandung
Hardware	ESP32-C3, ESP32 DevKit, MPU6050, RFM95W LoRa module (922 MHz)
Software	Arduino (C++), PHP, MySQL, Apache on an Ubuntu VPS
Method	Action research, with prototyping for the build

This is the project I built for my undergraduate thesis. It's a small wearable that an elderly person clips to their waist. If they fall, the device picks it up on its own. If they need help for another reason, they can press a button. Either way, a receiver in the caretakers' room starts beeping, and the event shows up on a web page the staff can open on their phones.

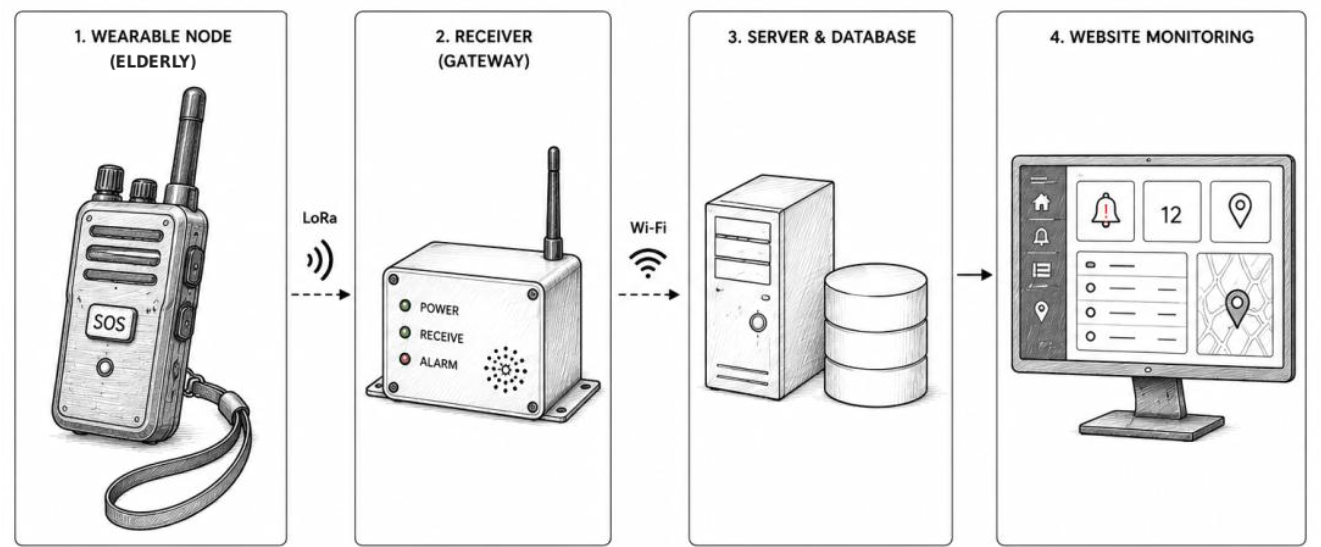


Figure 1. The four parts of the system: the wearable, the receiver, the server and the monitoring page.

Background

Panti Sosial Fakku Raqabah in Bandung has been caring for elderly people who have nobody else to look after them since 1986. At the time of this project it had 32 residents and 9 staff, and the residents live in separate rooms around the building. With that ratio, the staff can't keep an eye on everyone all the time.

Falls are the most common accident among older people. Muscles get weaker, balance and eyesight get worse, and reactions slow down, so a slip that a younger person would recover from can end in a fracture or a head injury. It gets much worse when nobody knows it happened, especially if the person can't get up or call for help.

That had already happened at the home. One of the caretakers told me about a resident who fell inside the building while the staff were in another area. Nobody noticed right away, so help came late. The home had no device for this kind of situation. A resident who needed help had to call out, or wait until someone came by.

So I set two goals for the project. The first was to detect falls automatically and get the alert to the staff without anyone having to do anything. The second was to give residents a button so they could ask for help themselves, even when they hadn't fallen.

Related work

Before building anything, I looked at similar projects. Most of them solved one part of the problem: some detected falls, some sent data over LoRa, some had a web dashboard. I didn't find one that put fall detection, an emergency button, LoRa and a dashboard into a single wearable, so that became the gap this project tries to fill.

Table 1. Features in related studies compared with this project.

Study	Fall detection	Emergency button	LoRa	Web dashboard
Tahir et al. (2022)	✓	–	–	–
Naeim et al. (2023)	✓	–	–	✓
Pandelaki et al. (2023)	✓	–	–	✓
Fitriandini et al. (2024)	✓	–	–	✓
Villa & Casilari (2025)	✓	–	✓	–
Tsani & Mulyadi (2019)	✓	–	–	–
Putra et al. (2025)	–	–	–	✓
Lousado et al. (2021)	–	–	✓	✓
This project	✓	✓	✓	✓

How the system works

There are three pieces of hardware and one web page (Figure 2). I go through each component in more detail on the next pages.

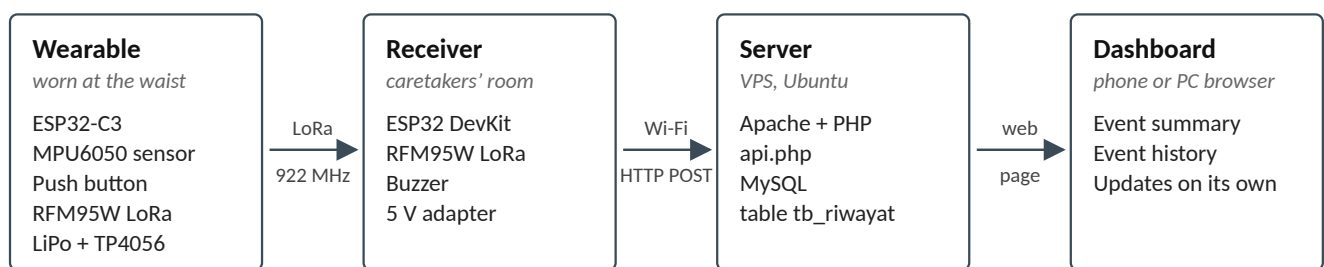


Figure 2. How the parts connect. Detection happens on the wearable, the receiver handles the alarm and the internet connection, and the server stores events and serves the dashboard.

Wearable. The wearable is designed like a small walkie-talkie with a clip on the back, so it can go on a belt or a waistband. All of the fall detection runs on the device, so it doesn't need Wi-Fi or a phone to work.

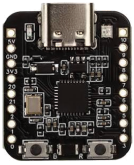
Receiver. The receiver stays in the caretakers' room and runs off a 5 V adapter, so battery life isn't a concern there. When a packet arrives, it reads it, sounds the buzzer and passes the event on to the server through the home's existing Wi-Fi. I put the internet connection on the receiver on purpose, so the battery-powered wearable only has to handle the low-power radio.

Server. The server is a small VPS (1 vCPU, 1 GB RAM, 60 GB SSD) running Ubuntu, Apache, PHP and MySQL. The receiver sends each event with an HTTP POST to a PHP script, `api.php`, which saves it in a table called `tb_riwayat`. The dashboard reads from the same table. Because it's on a VPS, the history is still there when my laptop is off, and the staff can open it from outside the home.

The components

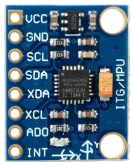
Here is every part I used and what it does. The first six are in the wearable. The receiver has its own RFM95W, plus an ESP32 DevKit and a buzzer.

In the wearable



ESP32-C3 microcontroller

This is the brain of the wearable. It reads the MPU6050, works out the magnitude, checks the button, and builds and sends the packet through the LoRa module. I chose it because the board is tiny, uses little power and already has the I²C and SPI buses that the sensor and the radio need. The chip has a single RISC-V core with Wi-Fi and Bluetooth LE built in, although the wearable doesn't use either of them.



MPU6050 motion sensor

A small module with an accelerometer and a gyroscope, both measuring along three axes (X, Y and Z). The accelerometer also picks up gravity, which is why it reads about 9.8 m/s² when the device is still. It can measure up to ± 16 g, so even a hard impact stays within its range. I picked it because it's small, cheap and used in a lot of fall detection research. For now I only use the accelerometer. Bringing in the gyroscope is on my list of improvements.



Push button emergency button

A plain switch that closes the circuit while it's held down, which the ESP32-C3 reads as a press. Pressing it sends a help request right away, without waiting for fall detection, which covers emergencies that aren't falls, like suddenly feeling unwell. In the case design it's big, red and on the front, so it's easy to find. Mechanical buttons also bounce a little, and that has to be handled so one press isn't counted as several.



RFM95W LoRa radio module

The ESP32 has no LoRa radio of its own, so this module adds one. It's built around Semtech's SX1276 chip, which uses spread-spectrum modulation, so a packet can still get through when the signal is weaker than the background noise. I use the 900 MHz version, set to 922 MHz, with an external antenna. The receiver has the same module, and both sides have to use the same spreading factor, bandwidth and sync word, or they can't hear each other.



LiPo battery 3.7 V power source

A light, thin rechargeable battery, which makes sense for something worn on the body all day. It gives about 3.7 V, and its capacity in mAh decides how long the wearable lasts between charges. Compared with regular lithium-ion cells, lithium polymer has a slightly higher energy density and can be made thinner, which helps keep the wearable small and light.



TP4056 charging board

A small board that charges the LiPo battery from a USB cable. It charges at a constant current first and then at a constant voltage, and stops by itself once the battery is full at 4.2 V, so the battery doesn't get overcharged. The version I used also has a protection circuit that keeps the battery from draining too far. Thanks to this board, the wearable can simply be recharged instead of needing new batteries.

In the receiver



ESP32 DevKit microcontroller

The controller in the receiver. It takes each packet from its RFM95W, reads the RSSI and SNR, sounds the buzzer and sends the event to the server over Wi-Fi. I used the full DevKit here because it has Wi-Fi built in and enough pins for the LoRa module and the buzzer, and on the receiver the size doesn't matter. It runs off a 5 V adapter, so it can stay on all the time without a battery.



Active buzzer 5 V alarm

It makes a sound as soon as current flows through it. An active buzzer has its own tone circuit inside, so the ESP32 only needs to switch it on and off instead of generating a signal. It's there so the staff hear an alert even when nobody is looking at the dashboard: double beeps for 30 seconds for a fall, and one long beep for the button.

Table 2 shows where each part connects on the two microcontrollers. The wiring diagrams are in Figures 4 and 5.

Table 2. *Pin connections.*

Wearable (ESP32-C3)			Receiver (ESP32 DevKit)		
MPU6050	SDA, SCL	GPIO8, GPIO9	RFM95W	SCK, MISO, MOSI	GPIO18, 19, 23
Push button	signal	GPIO1, internal pull-up	RFM95W	NSS, RST, DIO0	GPIO5, 14, 2
RFM95W	SCK, MISO, MOSI	GPIO4, 5, 6	Buzzer	+	GPIO27
RFM95W	NSS, RST, DIO0	GPIO7, 10, 3	5 V adapter	5V, GND	VIN, GND
TP4056	OUT+, OUT-	3V3, GND			

A note on “LoRaWAN”

The title says LoRaWAN, but the radio link itself is plain LoRa, sent point-to-point at 922 MHz. That's inside the 920 to 923 MHz band Indonesia allows for low-power devices without a special license, and the module transmits at 17 dBm, under the 20 dBm limit. What I took from LoRaWAN is the structure: end device, gateway, server. I left out the separate network server and the protocol features around it. For one building and one device they would only add cost and setup work, and a network server can still be added later if the system grows.

Table 3. *Which parts of LoRaWAN the system uses.*

LoRaWAN element	Used?	In this system
End device	Yes	The wearable, which sends the event
Gateway	Yes	The receiver, which forwards packets over Wi-Fi
Network server	Simplified	Its job is done by the receiver itself
Application server	Yes	The VPS with <code>api.php</code> , MySQL and the dashboard
Star topology	Yes	Wearables send to one receiver, then to one server
ISM band and Class A	Yes	Free 922 MHz band, radio only on while sending
Join, session keys, ADR	No	Devices use an ID in the packet, radio settings are fixed

The prototypes

Figure 3 shows the prototypes I tested with. Both are wired on a breadboard with jumper wires, without soldering. On the wearable, all the parts sit on one board so it can still be clipped to the waist.

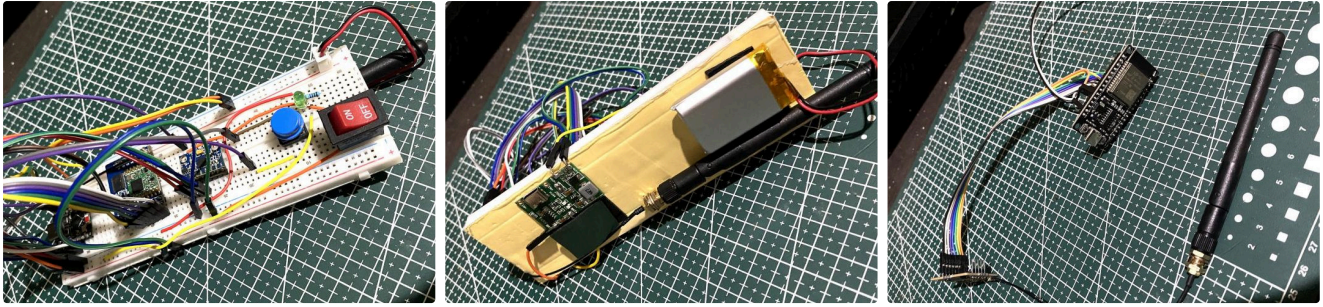


Figure 3. Left: the wearable, with the ESP32-C3, LoRa module, MPU6050, button, LED and power switch. Middle: its back, with the LiPo battery, TP4056 charger and antenna. Right: the receiver, with the ESP32 DevKit, LoRa module, antenna and buzzer.

Figures 4 and 5 show how everything is connected. On both devices the LoRa module uses SPI. In the wearable, the MPU6050 uses I²C and the button needs only one GPIO pin. In the receiver, the buzzer takes one GPIO pin and the board is powered from the 5 V adapter through VIN.

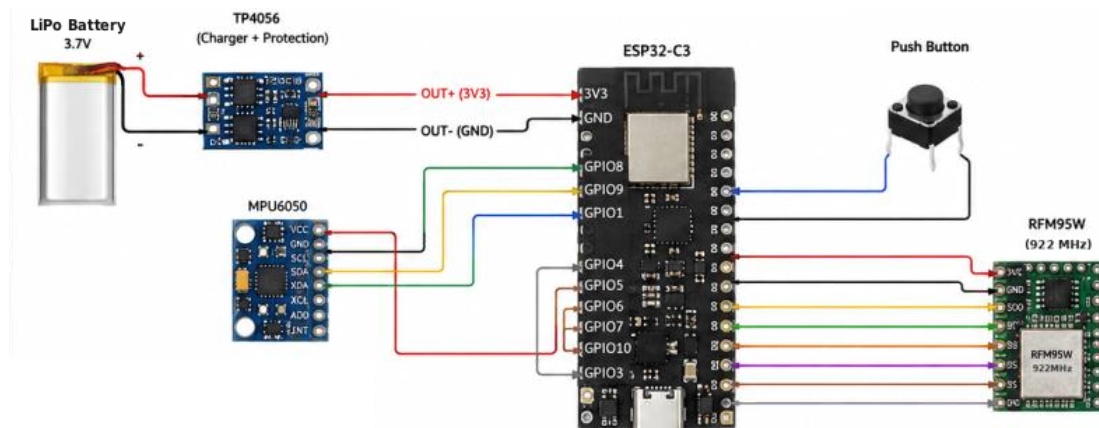


Figure 4. Wearable wiring. The battery powers everything through the TP4056.

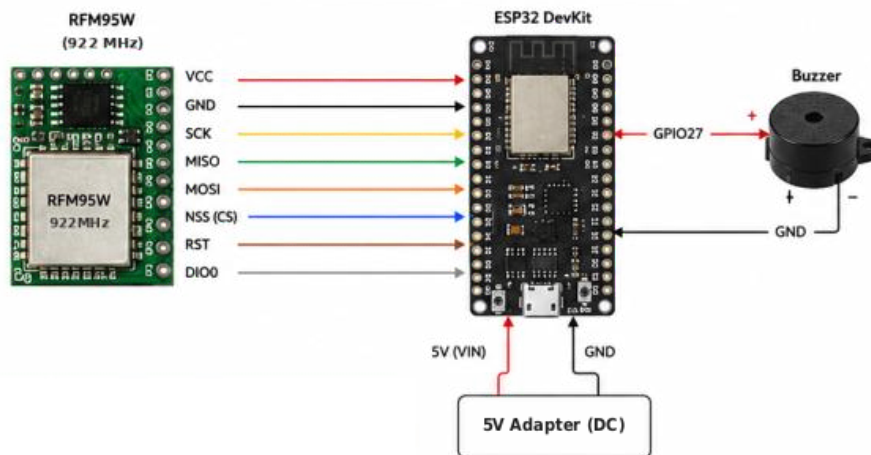


Figure 5. Receiver wiring. The ESP32 DevKit gets its power from the 5 V adapter.

Detecting a fall

The MPU6050 measures acceleration on the X, Y and Z axes. I combine the three into one number, the signal magnitude vector (SMV):

$$SMV = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

The nice thing about SMV is that it doesn't care which way the device is facing, which matters for something clipped to a belt that moves around. When the wearer is still, it reads about 9.8 m/s^2 , which is just gravity. During a fall there's usually a short dip while the body drops, then a sharp spike when it hits the floor. The device only looks at that spike. If SMV goes above 23.1 m/s^2 (about 2.36 g), it counts as a fall.

I didn't come up with 23.1 m/s^2 myself. It comes from Tsani and Mulyadi (2019), whose setup was the closest to mine: the same MPU6050, also worn at the waist, with 88% accuracy. Other studies used higher values, such as 3 g at the waist (Villa and Casilari, 2025) and 3.52 g on the chest (Bagalà et al., 2012), so this one is on the sensitive side.

I wear it on the waist because that's close to the body's centre of mass, so the sensor follows the whole body instead of every arm movement. Özdemir (2016) tested six positions on the body and got the best results at the waist.

A fixed threshold is about as simple as fall detection gets. I chose it because it runs easily on a microcontroller, needs no training data and gives an answer straight away. The downside is that some normal movements, like sitting down hard, can look like a fall. I come back to this at the end.

What the device sends

Every event is sent as a short text packet with three fields: device ID, event type and magnitude. For example, 1, JATUH, 58.25 means device 1 detected a fall (*jatuh* in Indonesian) with a peak of 58.25 m/s^2 . A button press is sent as TOMBOL. The ID is there so more wearables can be added later without mixing up their data.

When the receiver gets a packet, it also records the signal strength (RSSI) and the signal-to-noise ratio (SNR), then plays a buzzer pattern that depends on the event type. The two patterns sound different, so the staff know what happened without looking at a screen (Table 4).

There's no GPS in the wearable, but RSSI gives a rough idea of distance, since a weaker signal usually means the event happened farther from the receiver. I split it into three ranges (Table 5) and show the range next to each event on the dashboard. Over time, that could show which parts of the building see the most falls.

Table 4. Buzzer patterns on the receiver.

Event	Buzzer
Fall detected	Double beeps for 30 seconds
Button pressed	One long beep

Table 5. Rough range from RSSI.

Range	RSSI
Near	-75 dBm or stronger
Medium	-90 to -75 dBm
Far	Weaker than -90 dBm

These limits are starting values and still need tuning on site.

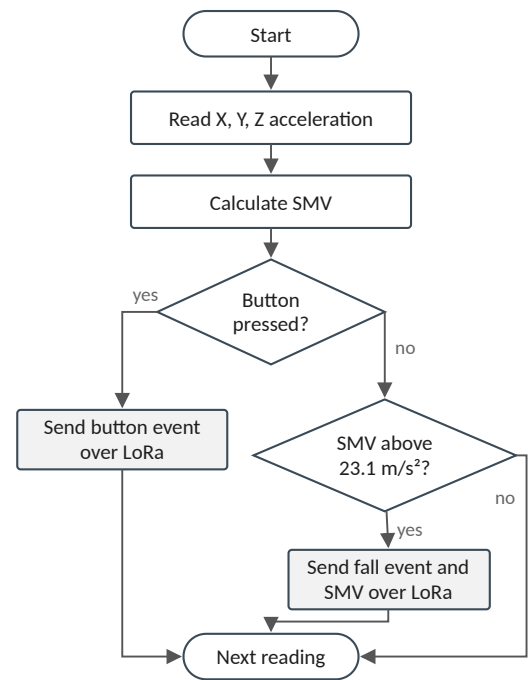


Figure 6. The loop on the wearable. The button is checked first, so a button press goes out immediately.

The dashboard

The dashboard is a single PHP page. The caretakers aren't technical, so I kept it plain: large text, everyday words and nothing to install. The top shows how many falls and button presses have been recorded and the latest event. Below that is the full history, newest first, and new events show up without reloading the page.

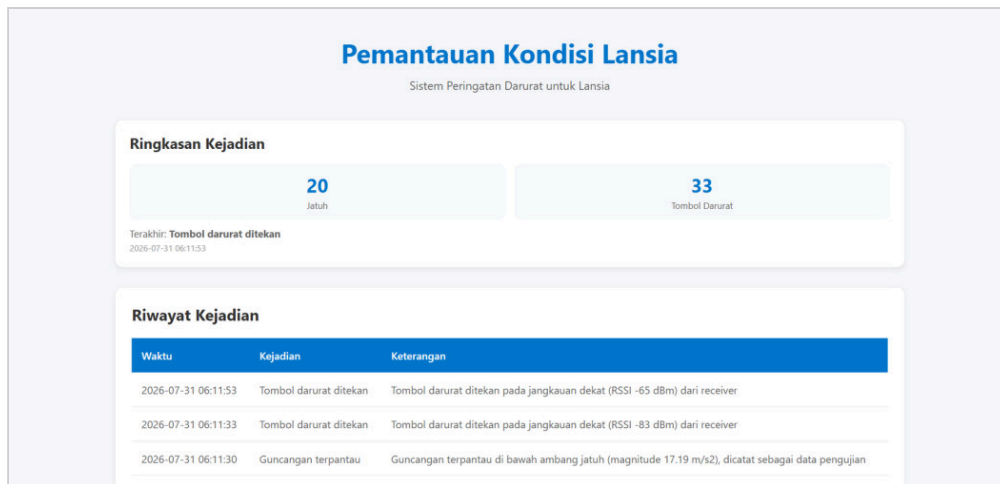


Figure 7. The monitoring page. It's in Indonesian because that's what the staff use: *Jatuh* means fall, *Tombol Darurat* means emergency button, and *Riwayat Kejadian* is the event history.

Testing

I tested one part at a time, from the sensor up to the whole chain, so that if something broke I'd know where. It was black-box testing: I only checked whether each part gave the right output for a given input.



Figure 8. Serial monitor output. Left: the wearable at rest (about 9.8), then dropped (58.25) and sending the packet. Top right: the receiver getting a fall (33.28, RSSI -52 dBm) and starting the 30-second alarm. Bottom right: a button press, with a magnitude of only 10.28 because nothing fell.

Table 6. Results for each part of the system.

Part	What I checked	Result
MPU6050 sensor	Magnitude follows the movement: about 9.8 at rest, 58.25 when dropped	Pass
LoRa link	The receiver got exactly what was sent: ID 1, fall, 33.28	Pass
Emergency button	The button event reached the receiver right after the press	Pass
Buzzer	30 seconds of beeping for a fall, one long beep for the button	Pass
Server	Every event saved with its time, type and description	Pass
Dashboard	History shows up, and new events appear without reloading	Pass

Range test

For the range test, the receiver stayed in one place and I took the wearable to 1, 5, 10 and 30 m away. At each distance I pressed the button three times and simulated three falls. For every trial I checked whether the receiver got the data, whether the alarm went off, and whether the event showed up on the dashboard. All 24 trials passed all three checks. Every room at the home is within 30 m of the caretakers' room, so this range covers the building.

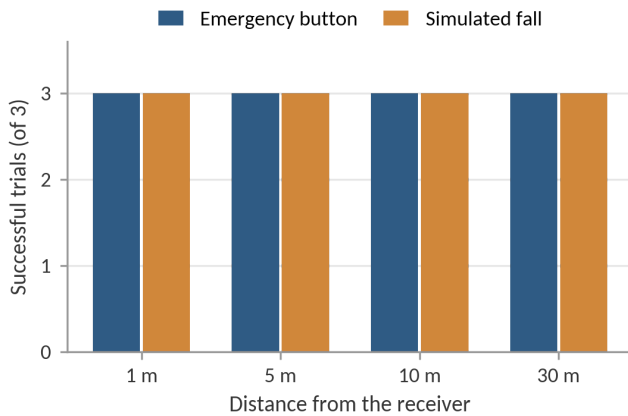


Figure 9. Successful trials at each distance.

Table 7. Range test results.

Distance	Button (m/s ²)	Fall (m/s ²)	Passed
1 m	10.28	51.30	6 of 6
5 m	10.35	56.10	6 of 6
10 m	10.61	99.11	6 of 6
30 m	11.42	108.50	6 of 6

The magnitudes come from the server log. They depend on how hard each movement was, not on the distance. I included them to show the data arrived intact.

So in the end, everything I set out to build worked together. Falls were detected and sent, the button worked, the data arrived over LoRa unchanged, and every event ended up on the dashboard.

Limitations

This is still a prototype, and the testing has some clear gaps. I only went up to 30 m, in an open area with no walls in between, so I don't know the real maximum range yet. Three trials per distance is a small sample. For safety, the falls were acted out by healthy adult volunteers, not by elderly residents. The threshold was borrowed from another study instead of being measured on my own device. And during testing, two packets arrived corrupted when the signal was weak, so the dashboard couldn't tell what kind of event they were.

What I would improve

The first thing I'd change is the threshold. It should be measured on this device, worn at the waist, instead of being taken from a paper. I'd also bring the gyroscope into the detection and check the body's angle after an impact. That should catch softer falls and stop quick sitting from being flagged as a fall.

For the radio, I want to test longer distances and through walls, and mount the antenna better so fewer packets get damaged. Further ahead, I'd like to turn it into a full LoRaWAN network that can cover more than one care home, add a heart-rate sensor, make the battery last longer, and send notifications straight to the caretakers' phones.

This is a shortened English version of my thesis, which was written in Indonesian under the title "Pembangunan Sistem Peringatan Darurat untuk Lansia Berbasis LoRaWAN dengan Fitur Deteksi Jatuh dan Tombol Darurat" (Informatics Engineering, Universitas Komputer Indonesia, 2026).